

Table of Contents

1. Planner and QueryGateway.....	1
1.1. Planner decisions	1
1.2. Public facade: QueryGateway	1
1.3. Examples end-to-end	2
1.4. REST endpoints (v1)	2
1.5. Row- and Field-Level Governance	2
1.6. Governed Aggregation Execution & Per-Stage Security Pushdown	3
1.6.1. Per-Stage Pushdown Mechanics	4
1.6.2. Configuration and Defaults	4
1.6.3. Request/Response: /api/query/plan.....	4
1.6.4. Request/Response: /api/query/find (Multi-Hop Aggregation).....	5
1.6.5. cURL Examples	5

Chapter 1. Planner and QueryGateway

The planner decides whether a query runs as a simple single-collection filter (FILTER mode) or as a MongoDB aggregation (AGGREGATION mode). This is transparent to callers.

In v1 the planner recognizes `expand(path)` and selects AGGREGATION mode when present. Otherwise, FILTER mode is used and the existing Morphia path is taken.

1.1. Planner decisions

- FILTER mode: No `expand(...)` in the query. The system generates a Morphia Filter and queries a single collection.
- AGGREGATION mode: One or more `expand(...)` directives are present. The system will build an aggregation pipeline (with `$lookup`, etc.). In v1, a placeholder pipeline is returned to establish the contract.

1.2. Public facade: QueryGateway

A new collection-agnostic facade exposes plan/inspect capabilities and will later execute aggregation pipelines when enabled.

Java

```
import com.e2eq.framework.model.persistent.morphia.planner.PlannerResult;
import com.e2eq.framework.model.persistent.morphia.query.QueryGateway;
import com.e2eq.framework.model.persistent.morphia.query.QueryGatewayImpl;

public class PlannerSnippet1 {
    static class Order {}
    public static void main(String[] args) {
        QueryGateway gateway = new QueryGatewayImpl();
        PlannerResult pr = gateway.plan("expand(customer) && status:active", Order.class);
        System.out.println(pr.getMode());           // AGGREGATION
        System.out.println(pr.getExpandPaths());    // [customer]
    }
}
```

If you prefer a lower-level API:

Java

```
import com.e2eq.framework.model.persistent.morphia.MorphiaUtils;
import com.e2eq.framework.model.persistent.morphia.planner.PlannedQuery;

PlannedQuery planned = MorphiaUtils.convertToPlannedQuery(
    "expand(items[*].product) && status:active",
    Order.class
);
```

```
switch (planned.getMode()) {
  case FILTER:
    // Use planned.getFilter() with Morphia datastore
    break;
  case AGGREGATION:
    // Use planned.getAggregation() to inspect the pipeline (placeholder in v1)
    break;
}
```

1.3. Examples end-to-end

- No expansion (FILTER mode):

```
q = "realm:acme && status:active && fields:[+_id,+total]"
```

- With expansion (AGGREGATION mode):

```
q = "expand(customer, fields:[+name,+tier]) && status:active &&
fields:[+_id,+customer.name]"
```

See also: [Relationship hydration with expand\(path\)](#). For concrete pipelines, see [Aggregation examples: what expand\(...\) compiles to](#).

1.4. REST endpoints (v1)

The gateway (`QueryGatewayResource`, `@Path("/api/query")`, `@FunctionalMapping(area="integration", domain="query")`) exposes a planner endpoint plus generic governed query operations:

- `POST /api/query/plan`—returns planner mode (`FILTER` vs `AGGREGATION`) and expand paths detected
- `POST /api/query/find`—executes governed `FILTER` and `AGGREGATION` queries, returning the standard `Collection<T>` envelope
- `POST /api/query/count`—returns the governed count for a `FILTER`-mode query
- `POST /api/query/delete`—deletes a single entity by id (governed; 404 across `DataDomain` boundaries)
- `POST /api/query/deleteMany`—deletes the matching `FILTER`-mode set
- `POST /api/query/export`—streams a governed CSV export

1.5. Row- and Field-Level Governance

All query gateway operations execute through multi-layered row- and field-level security filters derived from `RuleContext` and `DataDomainResolver`, failing closed when no valid security context is

established:

1. Root Entity Governance:

- `securedFilterArray(...)` ANDs the query filter with the caller's active `DataDomain` scope and rule `filterConstraints` resolved from `RuleContext`.
- Field-level exclusions (`applyExcludedFieldProjection(...)`) exclude sensitive fields at the MongoDB datastore level, preventing them from leaving MongoDB.

2. Fail-Closed Guarantee:

- Unless `SecurityContext.isIgnoringRules()` is explicitly set, requests lacking an established `PrincipalContext` and `ResourceContext` are immediately refused with HTTP 500 (`{"error": "SecurityContextMissing"}`).

3. Information Leakage Prevention:

- `delete` ANDs the target `_id` with the secured row filter; records outside the caller's `DataDomain` return HTTP 404 (`deletedCount=0, success=false`) without disclosing whether the entity exists in another tenant.

1.6. Governed Aggregation Execution & Per-Stage Security Pushdown

When a query contains `expand(...)` directives, the planner selects `AGGREGATION` mode and compiles the query into a multi-stage MongoDB aggregation pipeline via `MongoAggregationCompiler`:

flowchart TD

```
subgraph Root ["Root Document Stage"]
    RMatch["$match: dataDomain + Root Rule Constraints"]
    RFilter["$match: User Root Filter"]
end

subgraph Hop1 ["Hop 1: $lookup inner pipeline (e.g. orderRef)"]
    H1Match["$match: $expr eq [_id, $$ref_id]"]
    H1Sec["$match: Order DataDomain + Order Rule Constraints"]
    H1Proj["$project: Exclude Order Sensitive Fields"]
end

subgraph Hop2 ["Hop 2: $lookup inner pipeline (e.g. supplierRef)"]
    H2Match["$match: $expr eq [_id, $$ref_id]"]
    H2Sec["$match: Supplier DataDomain + Supplier Rule Constraints (Accumulated)"]
    H2Proj["$project: Exclude Supplier Sensitive Fields"]
end

Root --> Hop1
Hop1 --> Hop2
Hop2 --> FinalProj["Final $project / field filtering"]
```

1.6.1. Per-Stage Pushdown Mechanics

1. **Root Pre-Filtering:** The root collection pipeline begins with a leading `$match` enforcing the root model's `DataDomain` policy and security rules.
2. **Inner Pipeline `$lookup` Security:** For each expansion hop, target entity row-level constraints are pushed directly into the inner `pipeline: [ ... ]` of MongoDB's `$lookup` stage.
3. **Monotonic Restriction Accumulation:** Security restrictions accumulate along multi-hop paths (e.g. Customer → Order → Supplier). Downstream hops inherit and compound upstream access constraints, ensuring that traversal can never be exploited to elevate privileges or bypass intermediate isolation boundaries.
4. **Target Field Exclusion Pushdown:** Target entity field exclusions (`excludedFieldPaths`) are pushed into the inner pipeline as a `$project` stage, redacting sensitive fields directly within the MongoDB engine.
5. **Cross-Tenant Reference Pruning:** If a foreign reference points to an entity in a different tenant or an entity that fails target row security, the inner `$match` stage filters it out. The populated property resolves to an empty array or `null`, completely preventing cross-tenant data leakage.

1.6.2. Configuration and Defaults

Property	Type	Default	Description
<code>feature.queryGateway.execution.enabled</code>	boolean	<code>false</code>	Guards execution of <code>POST /api/query/find</code> for aggregation queries. When <code>false</code> , aggregation queries return HTTP 501 (<code>ExecutionDisabled</code>). Must be set to <code>true</code> to enable governed aggregation pipeline execution.
<code>feature.queryGateway.maxPageSize</code>	integer	<code>1000</code>	Maximum allowed page size (<code>limit</code>) for <code>/api/query/find</code> requests. Requests exceeding this limit are capped.



`count` and `deleteMany` explicitly reject `expand(...)` queries with HTTP 400 (`InvalidQuery`) because relational hydration is invalid for scalar counting and bulk document deletion.

1.6.3. Request/Response: `/api/query/plan`

Request body

```
{
  "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
  "query": "expand(orderRef) && expand(orderRef.supplierRef) && status:active"
}
```

Response body

```
{
  "mode": "AGGREGATION",
  "expandPaths": [
    "orderRef",
    "orderRef.supplierRef"
  ]
}
```

1.6.4. Request/Response: /api/query/find (Multi-Hop Aggregation)

Request body (AGGREGATION mode with multi-hop hydration)

```
{
  "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
  "query": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1",
  "page": { "limit": 10, "skip": 0 },
  "sort": [ { "field": "createdDate", "dir": "DESC" } ],
  "realm": "acme"
}
```

Response body (Collection envelope with hydrated nested entities)

```
{
  "offset": 0,
  "limit": 10,
  "rows": [
    {
      "id": "607f1f77bcf86cd799439001",
      "customerName": "Cust-1",
      "orderRef": {
        "id": "607f1f77bcf86cd799439002",
        "orderNumber": "ORD-1001",
        "supplierRef": {
          "id": "607f1f77bcf86cd799439003",
          "supplierName": "Acme Industrial Supplies"
        }
      }
    }
  ],
  "rowCount": 1,
  "filter": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1"
}
```

1.6.5. cURL Examples

Plan multi-hop expansion

```
curl -sS -X POST \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <jwt-token>' \
  localhost:8080/api/query/plan \
  -d '{
    "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
    "query": "expand(orderRef) && expand(orderRef.supplierRef) && status:active"
  }'
```

Execute governed multi-hop find

```
curl -sS -X POST \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer <jwt-token>' \
  localhost:8080/api/query/find \
  -d '{
    "rootType": "com.e2eq.framework.api.query.MultiHopCustomer",
    "query": "expand(orderRef) && expand(orderRef.supplierRef) && customerName:Cust-1",
    "page": { "limit": 10, "skip": 0 },
    "realm": "acme"
  }'
```

See also: [Relationship hydration with expand\(path\)](#) and [Query Language Reference](#).